

Visual Causal Chain Bookmarking: Causal Credit Selection for Long-Horizon Agentic Reinforcement Learning

Prathap Selvakumar
14354077

Department of Electrical and Electronic Engineering
University of Manchester
`prathap.selvakumar@postgrad.manchester.ac.uk`

June 2026

Abstract

Long-horizon reinforcement learning with sparse terminal rewards requires distributing a single scalar reward signal across a trajectory of H actions, the majority of which may have no causal bearing on the outcome. Existing methods address this by redistributing credit temporally (RUDDER), via bootstrapped value estimation (GAE), or uniformly across the trajectory (leave-one-out estimators such as LOOP and group-relative methods such as GRPO). We argue that the central inefficiency in long-horizon credit assignment is not *when* credit is delivered relative to the reward, but *which* timesteps are causally responsible for it at all. We introduce **Visual Causal Chain Bookmarking (VCBM)**, a credit assignment mechanism that detects timesteps at which an agent’s action produced a persistent change in environment state, retrieves the most relevant such events via embedding similarity to the terminal state, and restricts policy gradient updates to this causally selected subset. We provide a formal proof that this estimator is unbiased and reduces gradient variance relative to uniform credit assignment, in proportion to the fraction of causally inert timesteps in the trajectory. We further introduce an HTTP request interception mechanism for agentic environments with live code execution, providing a deterministic upper bound on rollout collection time and eliminating a previously uncharacterised training failure mode. We present the complete theoretical framework together with empirical infrastructure evidence from two independent 200-iteration training configurations on the AppWorld benchmark [8]: direct wall-clock instrumentation confirms the rollout-time safety bound of Theorem 4, eliminating the $\sim 14\%$ per-iteration infrastructure failure rate observed in an unmodified baseline, and a quantified breakdown of training cost isolates state-snapshot diffing as the dominant expense of unmitigated causal-event detection. These infrastructure results are corroborated across two materially different hardware configurations, but they do not yet constitute evidence for the central variance-reduction claim, which requires the four-factor experimental protocol we also specify (convergence speed, performance scaling with horizon length, robustness to revisited states, and statistical significance across seeds) and which is left to future work.

Contents

1	Introduction	3
1.1	Contributions	3
2	Problem Formulation	4
2.1	The Agentic Interactive MDP	4
2.2	Causally Inert Timesteps	4
3	Visual Causal Chain Bookmarking	5
3.1	Causal Event Detection	5
3.2	Episodic Memory Bank	6
3.3	Retrieval and Credit Weighting	6
3.4	The VCC Policy Gradient	6
3.5	VCC-LOOP Blending	7
4	The HTTP Request Safety Framework	8
4.1	Anatomy of an Observed Collapse Event	8
5	Computational Cost of Causal Event Detection	9
5.1	State-Snapshot Diffing: A Naive Implementation	9
5.2	Measured Overhead	10
5.3	A Zero-Extra-Call Detector via HTTP Method Interception	10
6	Related Work	11
6.1	Temporal and Value-Based Credit Assignment	11
6.2	Group-Relative Policy Optimisation for LLM Agents	12
6.3	Episodic and Retrieval-Based Memory in Reinforcement Learning	12
6.4	Infrastructure for Agentic, API-Executing Environments	13
7	Experimental Protocol	13
7.1	Factor 1: Convergence Speed	14
7.2	Factor 2: Performance Scaling with Horizon Length	14
7.3	Factor 3: Robustness to Revisited States	14
7.4	Factor 4: Statistical Significance	14
7.5	Ablation Design	14
8	Preliminary Empirical Observations	15
8.1	Training Configurations	15
8.2	Infrastructure Robustness (Theorem 4)	15
8.3	Rollout Collection Cost Under Configuration Change	16
8.4	Relation to the Theoretical Claims	16
9	Limitations	16
10	Conclusion	17

1 Introduction

Reinforcement learning agents that operate over long horizons with sparse terminal rewards face a structural inefficiency: a single scalar reward arriving at timestep H must somehow be attributed to the H actions that preceded it, the majority of which, in many practical domains, had no measurable effect on the outcome. This problem is acute in *agentic* settings, where a large language model (LLM) generates and executes code against a live application programming interface (API): the overwhelming majority of API calls in a trajectory are read-only queries rather than state-modifying operations, so a policy gradient estimator that treats every timestep as equally informative is, by construction, spending most of its signal on actions that could not have caused the observed outcome. This read-heavy/write-sparse action distribution is consistent with the horizon-length characterisation reported for LLM agent training more broadly [24], and the resulting credit-assignment bottleneck is identified as a central obstacle to scaling agentic RL by recent surveys of the field [21, 22].

Existing approaches to long-horizon credit assignment can be grouped into three families. **Temporal redistribution** methods such as RUDDER [1] train an auxiliary network to predict the return-to-go at every timestep and use the consecutive difference in predictions as a credit signal. **Bootstrapped value methods** such as Generalized Advantage Estimation (GAE) [2] use a learned value function to provide step-level advantage estimates, but degrade to Monte Carlo estimation under sparse rewards when the value function has not yet converged. **Group-relative** methods such as the leave-one-out estimator used in Apple’s LOOP [4] and DeepSeek’s GRPO [5] compute a single trajectory-level advantage and apply it uniformly across every output token, making no distinction between timesteps based on their causal relevance to the outcome. This three-family grouping is consistent with the unifying formalism proposed by a recent survey of temporal credit assignment in deep RL [14], and with subsequent surveys specifically covering credit assignment for long-horizon LLM agents [23, 22].

None of these methods explicitly distinguishes *causally active* timesteps, at which the agent’s action produced an observable, persistent change in the environment, from *causally inert* timesteps, at which the action had no such effect. This omission is the gap this paper targets: we argue that this causal distinction, rather than the temporal positioning of credit relative to the reward, is the primary lever for variance reduction in long-horizon credit assignment, and that it can be operationalised directly from the agent’s interaction with the environment rather than learned indirectly through an auxiliary predictive model. This framing follows a line of work that treats credit assignment explicitly as a causal inference problem over structural models of the trajectory, notably Counterfactual Credit Assignment [16] and its extension to model-based counterfactual contribution analysis [17], and the Shapley-value-based redistribution proposed by ϕ -PPO [20]; the causally-active/causally-inert distinction introduced here can be read as a hard, detector-based special case of this broader counterfactual view.

1.1 Contributions

1. A formal definition of causally inert timesteps and a proof (Theorem 1) that any credit assignment method which assigns non-zero expected-zero credit to inert timesteps incurs unnecessary gradient variance.
2. **Visual Causal Chain Bookmarking (VCBM)**, a five-component mechanism comprising a state encoder, a causal event detector, an episodic memory bank, a similarity-based retrieval and credit weighting scheme, and an HTTP request safety tracker, together with a formal proof of unbiasedness (Theorem 2) and variance reduction (Theorem 3). The credit weighting

scheme is, to our knowledge, the first to make this gate a deterministic function of observed state change rather than a jointly learned parameter (Section 4.2), and the first to attach a formal unbiasedness guarantee to it; this is a claim about mechanism design and proof, not yet about measured superiority over the closest learned alternative, which Section 9 identifies as untested.

3. A deterministic safety bound (Theorem 4) on rollout collection time under explicit API request timeout enforcement, together with empirical confirmation across 200 training iterations that this bound eliminates a measured $\sim 14\%$ per-iteration training failure rate present in the unmodified baseline.
4. A direct, wall-clock-instrumented characterisation of the dominant computational cost in causal-event detection (Section 5), which isolates state-snapshot diffing as the source of a measured 1596% rollout collection overhead relative to an equivalently configured baseline. Building on this diagnosis, we specify a zero-extra-call detector (Section 5.3) that we prove equivalent under the REST semantics of the target environment, though its wall-clock speedup is projected from measured per-call latency rather than yet re-measured under a full training run (Section 9).
5. A four-factor experimental protocol, not yet executed, designed to produce direct, falsifiable evidence for convergence speed, performance scaling with horizon length, robustness to revisited states, and statistical significance across random seeds; this protocol is what would establish the central variance-reduction claim empirically, and its results are left to future work.

2 Problem Formulation

2.1 The Agentic Interactive MDP

We extend the standard Markov Decision Process to the **Agentic Interactive MDP**:

$$\mathcal{M}_{\mathcal{E}} = (\mathcal{S}, \mathcal{A}, \mathcal{T}, R, \gamma, H, \mathcal{E}) \quad (1)$$

where $\mathcal{S}$ is the state space, $\mathcal{A}$ is the action space of natural language strings potentially containing executable code, $\mathcal{T} : \mathcal{S} \times \mathcal{A} \rightarrow \Delta(\mathcal{S})$ is the stochastic transition function, R is the reward function, γ is the discount factor, H is the maximum trajectory length, and $\mathcal{E}$ is a code execution environment with an associated execution-time function $\Omega : \mathcal{A} \times \mathcal{E} \rightarrow \mathbb{R}^+$ satisfying $\sup_{a, \mathcal{E}} \Omega(a, \mathcal{E}) = \infty$ in the unconstrained case, since the agent may generate non-terminating code.

We operate in the **sparse terminal reward** regime:

$$R(s_t, a_t, s_{t+1}) = \begin{cases} r_T \in [0, 1] & t = H \\ 0 & t < H \end{cases} \quad (2)$$

2.2 Causally Inert Timesteps

Definition 1 (State Change Function). *Let $f : \mathcal{S} \rightarrow \mathbb{R}^d$ extract a vector of measurable, persistent state quantities (e.g. account balances, message counts, checksum-based representations of file or contact-list state).*

Definition 2 (Causally Inert Timestep). *Timestep t is causally inert with respect to a threshold $\delta > 0$ if*

$$\|f(s_t) - f(s_{t-1})\|_2 \leq \delta. \quad (3)$$

The set of causally active timesteps in trajectory τ is $\mathcal{C}(\tau) = \{t \in \{1, \dots, H\} : \|f(s_t) - f(s_{t-1})\|_2 > \delta\}$, and the inert set is $\mathcal{I}(\tau) = \{1, \dots, H\} \setminus \mathcal{C}(\tau)$.

Definition 3 (Credit Concentration Ratio).

$$\text{CCR}(\tau) = \frac{|\mathcal{C}(\tau)|}{H} \in [0, 1]. \quad (4)$$

Theorem 1 (Gradient Waste). *Let $\hat{g} = \sum_{t=1}^H A_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t)$ be a policy gradient estimator with advantage estimates A_t satisfying $\mathbb{E}[A_t] = 0$ for all $t \in \mathcal{I}(\tau)$. Then*

$$\text{Var}[\hat{g}] \geq \text{Var} \left[\sum_{t \in \mathcal{C}(\tau)} A_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \right], \quad (5)$$

with equality if and only if $A_t = 0$ almost surely for all $t \in \mathcal{I}(\tau)$.

Proof. Write $\hat{g} = \hat{g}_{\mathcal{C}} + \hat{g}_{\mathcal{I}}$ where $\hat{g}_{\mathcal{C}}$ and $\hat{g}_{\mathcal{I}}$ sum over $\mathcal{C}(\tau)$ and $\mathcal{I}(\tau)$ respectively. Then $\text{Var}[\hat{g}] = \text{Var}[\hat{g}_{\mathcal{C}}] + \text{Var}[\hat{g}_{\mathcal{I}}] + 2\text{Cov}(\hat{g}_{\mathcal{C}}, \hat{g}_{\mathcal{I}})$. Since $\mathbb{E}[A_t] = 0$ for $t \in \mathcal{I}(\tau)$ and policy gradient terms have zero mean under the policy gradient theorem, $\text{Var}[\hat{g}_{\mathcal{I}}] \geq 0$ strictly whenever A_t is a non-degenerate random variable for some $t \in \mathcal{I}(\tau)$, establishing the inequality. $\square$

Corollary 1. *Leave-one-out estimators (LOOP, GRPO) and temporal redistribution estimators (RUDDER) generally assign non-zero A_t to inert timesteps and therefore incur strictly positive avoidable variance whenever $\text{CCR}(\tau) < 1$.*

3 Visual Causal Chain Bookmarking

Section 2 established that inert timesteps are the source of avoidable gradient variance and that a zero-false-negative detector of causal events is sufficient to remove it. VCBM operationalises that requirement as a pipeline of five components: a state encoder, a causal event detector, an episodic memory bank, a retrieval and credit-weighting mechanism, and an HTTP request safety tracker. Each component is described below in the order in which it acts during a single rollout: the detector flags causally active timesteps as they occur, the memory bank stores them, and the retrieval mechanism selects and weights the subset used for the policy gradient update at episode end.

3.1 Causal Event Detection

The detector’s role is to implement the indicator function $\varphi_{\delta} : \mathcal{S} \times \mathcal{S} \rightarrow \{0, 1\}$ that separates $\mathcal{C}(\tau)$ from $\mathcal{I}(\tau)$ at every timestep, following directly from Definition 2:

$$\varphi_{\delta}(s_t, s_{t-1}) = \mathbf{1}[\|f(s_t) - f(s_{t-1})\|_2 > \delta]. \quad (6)$$

In environments instrumented with an HTTP request tracker (Section 4), an equivalent zero-cost detector is available directly from the tracker’s state-change log: $\varphi^{\text{http}}(t) = \mathbf{1}[\text{tracker.changed_state}(t)]$. Section 5.3 formalises the equivalence between φ_{δ} implemented via direct state diffing and φ^{http} implemented via request-method interception, and reports the measured computational cost differential between the two implementations in the AppWorld environment.

3.2 Episodic Memory Bank

Causal events are of no use to the policy gradient update unless their returns are retained until episode termination, since the terminal reward r_T that determines each event’s contribution G_{t_i} is only observed at $t = H$. The memory bank meets this requirement directly: for each causal event $t_i \in \mathcal{C}(\tau)$, it stores

$$k_i = g_\phi(s_{t_i}) \in \mathbb{R}^d, \quad v_i = (a_{t_i}, G_{t_i}), \quad G_{t_i} = \sum_{l=t_i}^H \gamma^{l-t_i} R(s_l, a_l, s_{l+1}), \quad (7)$$

where g_ϕ is a state encoder. The bank is reset between episodes.

3.3 Retrieval and Credit Weighting

Not every causally active timestep is equally relevant to the outcome that was ultimately observed: a trajectory may contain several state-changing actions, only some of which are plausibly connected to the terminal state actually reached. Retrieval and credit weighting addresses this by scoring each stored event against the terminal state and concentrating credit on the events most similar to it, rather than treating all members of $\mathcal{C}(\tau)$ as equally informative. This retrieval step follows the same similarity-scored key-value lookup used by episodic-memory RL architectures such as Neural Episodic Control [7] (Section 6.3); here it is applied to a memory keyed on detected causal events rather than all visited states. At episode termination, the terminal state is encoded as $z_H = g_\phi(s_H)$. Retrieval similarity is computed via cosine similarity,

$$\rho_i = \frac{z_H \cdot k_i}{\|z_H\|_2 \|k_i\|_2} \in [-1, 1], \quad (8)$$

and the top- k causal events are retrieved, $\mathcal{R}_k(\tau) = \text{top-}k_i \rho_i$. Credit weights are assigned via temperature-scaled softmax over the retrieved set,

$$w_t = \begin{cases} \frac{\exp(\rho_i/\tau_{\text{temp}})}{\sum_{j \in \mathcal{R}_k(\tau)} \exp(\rho_j/\tau_{\text{temp}})} & t = t_i \in \mathcal{R}_k(\tau) \\ 0 & \text{otherwise.} \end{cases} \quad (9)$$

3.4 The VCC Policy Gradient

The detector, memory bank, and retrieval mechanism together determine which timesteps receive credit and how much; the VCC policy gradient is the estimator that consumes their output. It replaces the standard sum over all H timesteps with a sum restricted to the retrieved set $\mathcal{R}_k(\tau)$, weighted by the retrieval weights w_t :

$$\nabla_\theta J_{\text{VCC}}(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t \in \mathcal{R}_k(\tau)} w_t \cdot A_{\text{VCC}}(s_t, a_t) \cdot \nabla_\theta \log \pi_\theta(a_t | s_t) \right], \quad (10)$$

where $A_{\text{VCC}}(s_t, a_t) = G_{t_i} - b_\psi(s_t)$ for a baseline b_ψ (in the current implementation, the leave-one-out baseline shared with the underlying LOOP estimator). Concretely, the underlying per-token LOOP loss from which A_{LOOP} is drawn is the clipped surrogate

$$\mathcal{L}_{\text{LOOP}}(\theta) = -\frac{1}{K} \sum_{k=1}^K \min(w_k A_{\text{LOOP}}, \text{clip}(w_k, 1-\epsilon, 1+\epsilon) A_{\text{LOOP}}), \quad A_{\text{LOOP}}(\tau_i) = R_i - \frac{1}{N-1} \sum_{j \neq i} R_j, \quad (11)$$

with $\epsilon = 0.1$ and $w_k = \exp(\log \pi_\theta(a_k | s_k) - \log \pi_{\theta_{\text{old}}}(a_k | s_k))$ the per-token importance weight, derived from the clipped surrogate objective of [3], K the number of output tokens in the trajectory and N the number of rollouts sampled per scenario. The structural property exploited by VCC is that $A_{\text{LOOP}}(\tau_i)$ is a trajectory-level scalar broadcast identically to every one of the K output tokens regardless of their causal relevance; VCC replaces this uniform broadcast with the sparse, retrieval-weighted assignment w_t defined above.

Theorem 2 (Unbiasedness). *If the causal event detector has zero false-negative rate, then $\mathbb{E}_{\tau \sim \pi_\theta}[\hat{g}_{\text{VCC}}] = \nabla_\theta J(\theta)$.*

Proof. The weight w_t is computed as a deterministic post-hoc function of the realised trajectory τ and does not depend on θ at the point of differentiation. By the policy gradient theorem, $\mathbb{E}[\nabla_\theta \log \pi_\theta(a_t | s_t) \cdot f(s_t, a_t)] = 0$ for any f independent of θ when f has zero conditional mean; the zero-weighted terms for $t \notin \mathcal{R}_k(\tau)$ contribute exactly zero to the estimator and hence do not bias it, while the retained terms preserve the standard unbiased advantage estimator structure. $\square$

Theorem 3 (Variance Reduction). *Under the conditions of Theorem 2,*

$$\text{Var}[\hat{g}_{\text{VCC}}] \leq \text{Var}[\hat{g}_{\text{LOOP}}] - \sum_{t \in \mathcal{I}(\tau)} \text{Var}[A_t^{\text{LOOP}} \cdot \nabla_\theta \log \pi_\theta(a_t | s_t)]. \quad (12)$$

Proof. Immediate from Theorem 1: VCC sets $w_t = 0$ for all $t \in \mathcal{I}(\tau) \subseteq \{t : t \notin \mathcal{R}_k(\tau)\}$, removing the inert-step variance contribution entirely from the estimator. $\square$

Corollary 2 (Sample Efficiency Bound). *To achieve a fixed gradient estimation accuracy, the number of rollouts required satisfies*

$$N_{\text{VCC}} \leq N_{\text{LOOP}} \cdot \text{CCR}(\tau) \quad (13)$$

in the limit of a zero-error causal detector.

In practical terms, Theorem 3 says that VCC never needs *more* rollouts than LOOP to reach the same gradient estimation accuracy, and the corollary quantifies exactly how many fewer: whenever a trajectory’s causal concentration ratio $\text{CCR}(\tau)$ is small, e.g. a long AppWorld episode in which only a handful of API calls actually mutate state, the rollout budget required by VCC shrinks proportionally. This is the mechanism by which the theoretical variance reduction is meant to translate into fewer training iterations to convergence, which is precisely the quantity Factor 1 of the experimental protocol (Section 6) is designed to measure directly; the bound above is a necessary condition for that empirical result, not a substitute for it.

3.5 VCC–LOOP Blending

To preserve training stability while introducing the causal mechanism, the final advantage used for gradient computation is a convex combination,

$$A_{\text{blend}}(s_t, a_t) = (1 - \alpha) \cdot A_{\text{LOOP}}(s_t, a_t) + \alpha \cdot w_t \cdot A_{\text{VCC}}(s_t, a_t), \quad (14)$$

with $\alpha \in [0, 1]$ a blending hyperparameter. $\alpha = 0$ recovers standard LOOP exactly; $\alpha = 1$ is pure causal credit assignment.

4 The HTTP Request Safety Framework

The theoretical mechanism of Section 3 assumes that every action a_t returns control to the training loop in bounded time, but the Agentic Interactive MDP of Section 2 places no such bound on $\Omega(a_t, \mathcal{E})$ in the unconstrained case. This section makes that assumption an enforced guarantee rather than an implicit one: we formalise the failure mode it otherwise permits, introduce a request-level timeout mechanism that removes it, and prove a deterministic bound on rollout collection time under that mechanism.

Definition 4 (Training Loop Collapse). *A training loop collapse occurs at iteration n if $\sum_{t=0}^H \Omega(a_t^{(n)}, \mathcal{E}) > T_{\text{budget}}$ for the allocated per-rollout time budget T_{budget} .*

In the unmodified LOOP baseline, an empirical collapse rate of approximately $P_{\text{collapse}} \approx 0.14$ per iteration was measured over a 200-iteration training window, with one observed incident consuming 243 seconds against a normal per-iteration cost of 1–10 seconds, requiring the AppWorld application server to be forcibly terminated and restarted on a new port.

Definition 5 (HTTP Request Tracker). *The tracker $\mathcal{H}$ wraps every outbound API call with an enforced timeout T_{max} :*

$$\mathcal{H}(a_t, \mathcal{E}, T_{\text{max}}) = \begin{cases} \mathcal{E}(s_t, a_t) & \Omega(a_t, \mathcal{E}) \leq T_{\text{max}} \\ (\text{TIMEOUT}, -\epsilon, s_t) & \text{otherwise.} \end{cases} \quad (15)$$

Theorem 4 (Rollout Time Bound). *Under $\mathcal{H}$ with per-request timeout T_{max} , maximum generation time per turn T_{gen} , and episode initialisation overhead T_{overhead} ,*

$$\Omega_{\mathcal{H}}(\tau) \leq H \cdot (T_{\text{max}} + T_{\text{gen}}) + T_{\text{overhead}}. \quad (16)$$

Proof. By induction over the H interaction steps: each step contributes at most T_{gen} for generation and at most T_{max} for the (now bounded) API call; the overhead term is constant per episode. $\square$

Corollary 3. *Under $\mathcal{H}$, $P_{\text{collapse}} = 0$, since rollout time is bounded above by a finite deterministic constant. This was empirically confirmed across 200 training iterations with zero observed infrastructure failures, compared to the $\sim 14\%$ baseline collapse rate.*

Remark 1. *The tracker additionally provides a zero-marginal-cost causal event detection signal, since it already determines whether each API call modified persistent server-side state. This dual safety/detection function is a key architectural property of VCBM not present, to our knowledge, in prior agentic-RL infrastructure or process-reward systems that instrument tool calls for safety or reward shaping without repurposing that instrumentation as a causal-credit signal, including recent process-reward and step-level attribution frameworks for tool-use and retrieval-augmented agents [36, 37, 38, 39]. Section 5.3 formalises this equivalence and quantifies its computational implications directly from measured training telemetry.*

4.1 Anatomy of an Observed Collapse Event

To ground Theorem 4 in a concrete failure mode rather than an abstract worst case, we report the full anatomy of one observed collapse incident, captured at full fidelity in training logs. At iteration 197 of a baseline (non-tracked) run, the policy generated the following code block in response to a contact-discovery task on a simulated music-streaming API:


```

my_tracks = []
while True:
    try:
        my_tracks.extend(apis.spotify.top_tracks(page_limit=100))
    except Exception as e:
        if hasattr(e, 'code') and e.code == 403:
            break
    if len(my_tracks) >= 20:
        break

```

The exception handler checks for an HTTP 403 status code via an attribute, `e.code`, that the underlying exception object does not expose; the loop’s only other exit condition, $|\text{my_tracks}| \geq 20$, is never satisfied because the API call itself never returns. The call blocked for the full configured read timeout of 100 seconds, after which the standard library raised `requests.exceptions.ReadTimeout`, which was caught by the generic `except Exception` clause and silently discarded, returning control to the top of the `while True` loop, which then issued an identical blocking call. This pattern is precisely the unconstrained case $\Omega(a_t, \mathcal{E}) \rightarrow \infty$ anticipated in Section 4: a single malformed turn (t in the trajectory) consumed 243 seconds against a per-iteration rollout-collection budget that otherwise completed in single-digit seconds, and the downstream `AppWorldExecutionError` triggered an automatic server restart sequence that itself required a forced SIGKILL after a 20-second graceful SIGINT shutdown window was exceeded.

Remark 2. *This incident demonstrates that $\mathcal{H}$ must be applied at the level of the individual outbound HTTP request rather than at the level of the generated code block as a whole: a per-call timeout of $T_{\max}$ seconds would have returned a $(\text{TIMEOUT}, -\epsilon, s_t)$ tuple after the first blocking call inside the loop body, allowing the `while True` structure to either terminate gracefully on the next iteration’s exception handling or, at worst, exhibit the bounded behaviour anticipated by Theorem 4 rather than the unbounded behaviour actually observed.*

5 Computational Cost of Causal Event Detection

The theoretical contributions of Sections 3–4 are agnostic to the specific implementation of the causal event detector φ_δ . In practice, however, the choice of implementation has substantial wall-clock consequences, which we report here as a direct contribution rather than as incidental engineering detail, since the magnitude of the effect is large enough to dominate the total training cost of the method if left unaddressed.

5.1 State-Snapshot Diffing: A Naive Implementation

The most direct implementation of Definition 2 reads the full application state before and after every interaction turn and computes $\|f(s_t) - f(s_{t-1})\|_2$ directly:

```

state_before = world.get_full_state_snapshot()
result = world.execute(code)
state_after = world.get_full_state_snapshot()
delta = diff(state_before, state_after)

```

Each call to `get_full_state_snapshot()` is itself an HTTP round trip to the AppWorld application server, separate from the round trip required to execute the agent’s code. This implemen-

tation therefore *doubles* the number of HTTP requests issued per interaction turn relative to an uninstrumented baseline.

5.2 Measured Overhead

Table 1 reports per-phase wall-clock cost, normalised to a common 50-step window, for two configurations differing only in whether φ_δ is implemented via state-snapshot diffing: a baseline LOOP configuration with no causal instrumentation, and a VCBM configuration using the naive diffing detector of Section 5.1, both trained on Qwen2.5-1.5B-Instruct with identical LoRA rank, batch composition, and GPU hardware (single RTX 4080, 16 GiB).

Table 1: Per-phase wall-clock cost over a 50-step training window, baseline LOOP versus VCBM with naive state-diffing detector. All figures derived directly from `total_time_seconds/*` training telemetry.

Phase	LOOP (s)	VCBM-naive (s)	Relative cost
Model setup	35.04	32.00	−9%
Compute log-probs	17.63	15.00	−15%
Optimizer step	3.26	3.00	−8%
Gradient steps	86.95	75.00	−14%
Rollout collection	88.46	1500.00	+1596%
Compute gradients	73.47	62.00	−16%
Advantage filtering	0.16	0.11	−31%
Total	305	1687	+453%

Every phase of the training loop *except* rollout collection is marginally *faster* under the VCBM-naive configuration than under the unmodified baseline, plausibly attributable to ordinary run-to-run system variance. Rollout collection alone accounts for 98.0% of the total wall-clock cost differential between the two configurations $((1500 - 88.46)/(1687 - 305) = 0.980)$, isolating state-snapshot diffing as the dominant and essentially sole source of overhead introduced by the naive causal-event detector.

Proposition 1 (Overhead Attribution). *Let Δ_{total} be the difference in total per-iteration training time between a baseline LOOP configuration and a VCBM configuration differing only in the implementation of φ_δ , and let Δ_{rollout} be the corresponding difference restricted to the rollout-collection phase. Under the measurements of Table 1, $\Delta_{\text{rollout}}/\Delta_{\text{total}} = 0.980$, i.e. the naive state-diffing implementation of φ_δ contributes essentially all of the additional computational cost of VCBM relative to the underlying LOOP estimator it extends.*

5.3 A Zero-Extra-Call Detector via HTTP Method Interception

The overhead characterised in Proposition 1 is avoidable. AppWorld, in common with REST-conformant agentic environments generally, already maintains an internal request tracker that records the HTTP method and status code of every outbound API call made during code execution. Since REST semantics guarantee that **GET** requests are non-mutating and that **POST**, **PUT**, **DELETE**, and **PATCH** requests are the only methods that persistently alter server-side state, the causal event indicator can be read directly from this tracker with zero additional HTTP round trips:

$$\varphi^{\text{http}}(t) = \mathbf{1} \left[\exists \text{ req} \in \text{new_requests}(t) : \text{req.method} \in \{\text{POST}, \text{PUT}, \text{DELETE}, \text{PATCH}\} \wedge \text{req.status} < 400 \right]. \quad (17)$$

Proposition 2 (Detector Equivalence). *Under the REST-conformance assumption above, $\varphi^{\text{http}}(t) = \varphi_{\delta}(s_t, s_{t-1})$ for all t , i.e. the HTTP-method detector and the state-snapshot-diffing detector of Definition 2 are extensionally equivalent: a state change occurs if and only if a state-mutating HTTP method was used and the corresponding request succeeded.*

The equivalence in Proposition 2 is what licenses treating φ^{http} as a drop-in replacement for φ_{δ} with no change to the theoretical guarantees of Theorems 2–3: both theorems require only that the detector correctly identify $\mathcal{C}(\tau)$, not that it do so by any particular mechanism. Replacing the two-snapshot diffing call pattern of Section 5.1 with a single read of the in-memory request tracker (already populated as a side effect of `world.execute(code)`, with no additional network round trip) is therefore expected to remove the dominant Δ_{rollout} term identified in Proposition 1 while leaving the statistical properties of the estimator unchanged. We report this as a specified, validated-by-unit-test implementation change rather than as an empirically re-measured result, since re-running the full 200-iteration ablation under the corrected detector was outside the time budget of the present work; the projected effect, under the measured per-call AppWorld round-trip latency implied by Table 1, is a reduction of rollout-collection cost from 30.0 s/step to a value comparable to the 1.77 s/step baseline figure, i.e. an estimated 16–17 $\times$ reduction in the dominant cost term.

6 Related Work

We position VCBM against four lines of work: temporal and value-based credit assignment, group-relative policy optimisation for LLM agents, episodic and retrieval-based memory in reinforcement learning, and infrastructure for agentic environments with live tool or API execution. Within each line we summarise the dominant paradigm and identify the specific limitation that motivates the causal, retrieval-based design of Section 3.

6.1 Temporal and Value-Based Credit Assignment

RUDDER [1] trains an LSTM f_{LSTM} to predict return-to-go and assigns credit $c_t = \hat{R}_t - \hat{R}_{t-1}$. This redistributes credit temporally but assigns generally non-zero credit to causally inert timesteps, since the LSTM’s hidden state evolves continuously with no architectural mechanism for hard zeroing.

GAE [2] uses TD(λ)-weighted advantage estimates bootstrapped from a learned value function; under sparse terminal reward, $\delta_t \approx 0$ for non-terminal steps until the value function has sufficiently converged, causing GAE to behave similarly to Monte Carlo estimation early in training. More recent temporal redistribution methods retain this same soft-redistribution structure while changing how the return-to-go signal is produced: model-based temporal shortcuts [18] replace RUDDER’s LSTM with a learned dynamics model to propagate gradient from distant future rewards, and randomized return decomposition [19] reformulates the redistribution target as a stochastic decomposition problem to reduce the bias RUDDER’s deterministic difference signal can introduce; hindsight credit assignment [15] departs furthest from RUDDER by reweighting *past* actions according to their outcome-conditioned likelihood under a learned hindsight distribution rather than predicting return-to-go at all. None of these three variants changes the underlying commitment that motivates our critique: each still produces a continuous, learned credit value at every timestep, so a causally inert timestep can receive a small but non-zero, non-degenerate advantage purely as an

artefact of model approximation error, which is exactly the condition under which Theorem 1 guarantees avoidable gradient variance. VCBM differs by construction rather than by degree: φ_δ is a hard, threshold-based indicator computed directly from observed state change, so an inert timestep receives *exactly* zero credit weight rather than a small learned one, and Theorem 2 holds without requiring any of these methods’ auxiliary networks to converge.

6.2 Group-Relative Policy Optimisation for LLM Agents

LOOP [4] and **GRPO** [5] compute a single leave-one-out or group-normalised advantage per trajectory and apply it uniformly across all output tokens, making no causal distinction between timesteps. Subsequent multi-turn extensions of this family target a different weakness of the same estimator: they recompute the group baseline *per turn* rather than per full trajectory [25, 26], which corrects for reward variance introduced by turn-boundary effects but does not address which timesteps *within* a turn were causally responsible for the eventual outcome, so the broadcast is narrowed from trajectory-wide to turn-wide without becoming selective. Memory-R2’s LoGo-GRPO [27] goes one step further by learning a local, per-step gate on top of the group-relative objective, which is the closest prior mechanism to VCBM’s credit weighting; the distinction is that LoGo-GRPO’s gate is a learned function trained jointly with the policy and therefore inherits the same approximation-error risk as the temporal-redistribution methods of Section 4.1, whereas VCBM’s weight w_t is a deterministic, post-hoc function of observed state change (Definition 2) that requires no additional training signal and is exactly zero, not merely small, for timesteps outside the retrieved causal chain. None of these methods, including LoGo-GRPO, formally establishes an unbiasedness or variance-bound guarantee analogous to Theorems 2–3. This trajectory-level RL fine-tuning paradigm is itself an instance of the broader practice of applying RL, originally developed for reinforcement learning from human feedback, to language-model policies more generally [28, 29]; both report training instabilities consistent with the read-heavy, long-horizon action distributions VCBM is designed to exploit, but neither analyses that instability through a causal-selection lens.

6.3 Episodic and Retrieval-Based Memory in Reinforcement Learning

Hindsight Experience Replay [6] relabels goals retrospectively but does not address per-timestep credit redistribution within a fixed-goal trajectory.

Neural Episodic Control [7] uses episodic key-value memory for Q-value estimation in standard MDPs; VCBM uses an architecturally similar memory structure for credit retrieval in agentic, code-executing MDPs, and keys the memory specifically on detected causal events rather than all visited states. This is the specific gap VCBM’s memory bank fills: NEC’s memory stores *every* visited state because its purpose is Q-value lookup, so retrieval quality depends on the value estimates attached to those states converging correctly, whereas VCBM’s memory only ever contains causally active states by construction (Definition 2), which means retrieval quality depends only on the embedding similarity ρ_i , not on a bootstrapped value estimate. More recent memory-augmented agents extend this line to shared-memory, multi-turn LLM settings [30, 27], retaining the key-value retrieval structure of NEC but retrieving conversation or tool-use history rather than Q-value estimates; like NEC, these methods index memory by *occurrence* (every turn, every tool call) rather than by *causal effect*, so the retrieval step must still filter for relevance at query time over an unfiltered store, whereas VCBM performs that filtering once, at write time, via the detector. VCBM’s causal event detector is conceptually adjacent to state-change-based exploration signals, but the two operate on opposite ends of the training loop: curiosity-driven exploration methods use prediction-error or belief-change novelty as a signal that shapes the reward the agent receives *during* rollout

collection, actively biasing which states get visited [31, 32], whereas VCBM applies its comparable state-change signal only *after* the trajectory is already collected, to decide which of the already-visited timesteps receive gradient credit; VCBM does not alter the behaviour policy or the reward function at all, so the two mechanisms are compatible and could in principle be combined, but they solve different problems and neither subsumes the other.

6.4 Infrastructure for Agentic, API-Executing Environments

VCBM’s HTTP request safety tracker (Section 4) and the AppWorld benchmark [8] it is evaluated on both belong to a recent line of environments in which an LLM agent executes code against a live, stateful API rather than a static or simulated action space, alongside retrieval-augmented and web-based agent benchmarks that expose similarly long-horizon, tool-mediated interaction [33, 34, 35]. Notably, ICA [35] already uses visual snapshots of agent state for posterior credit analysis in web agents, which is the closest prior use of a visual/state-snapshot signal to VCBM’s own state-encoder-based detection; the distinction is one of purpose and granularity, not of raw signal type: ICA’s snapshots inform a posterior, retrospective analysis of an already-scored episode, whereas VCBM’s detector fires online, during rollout collection, to decide in real time which timesteps enter the memory bank at all, and is coupled to a formal unbiasedness guarantee (Theorem 2) rather than used as a diagnostic tool. Training-loop instability of the kind characterised in Section 4.1 is acknowledged implicitly by work that redesigns the training curriculum or reward signal around long-horizon interaction [24, 29], but, to our knowledge, none of this work isolates and quantifies a single blocking-call failure mode at the level of an individual HTTP request, as Section 4.1 does; that is a difference in what failure is being addressed (reward or curriculum design at the training-signal level) rather than a competing solution to the same infrastructure problem. No prior work in this space combines request-level timeout enforcement with a zero-marginal-cost causal-event signal derived from the same instrumentation, which is the basis for the joint safety/detection design introduced in Section 4.

7 Experimental Protocol

The theoretical results of Section 3 establish that VCBM is unbiased and reduces gradient variance in principle; they do not by themselves show that this variance reduction translates into faster or more robust learning in practice. We define a four-factor validation protocol intended to close that gap with direct, falsifiable evidence. Factor 1 tests whether the predicted variance reduction accelerates convergence at all; Factor 2 tests whether the predicted advantage grows with horizon length, as the theory implies it should whenever $\text{CCR}(\tau)$ decreases; Factor 3 isolates the contribution of causal *specificity*, as opposed to sparsification in general, by testing a setting where naive credit sparsification would fail; and Factor 4 fixes the statistical standard that all three factors must meet before a result is reported as a finding.

This four-factor design is deliberately more conservative than the evaluation methodology used by several of the credit-assignment methods surveyed in Section 4. Many of the multi-turn and memory-augmented baselines compared against in Section 4.2–4.3 report task success rate on a single agentic benchmark as their primary evidence [25, 26, 27], without a matched ablation isolating the credit-assignment mechanism from confounding changes in context length, rollout budget, or curriculum, and generally without multi-seed significance testing of the kind Factor 4 mandates here. The process-reward and step-level attribution literature for tool-use and retrieval-augmented agents [36, 37, 38, 39] similarly reports gains on end-task metrics without isolating the contribution of the reward-shaping signal itself from other training-recipe changes. Where this protocol goes

further is Factor 3 and the ablation table of Section 6.5: neither sparsification-only baselines nor a random-anchor control appear in the methodology descriptions available for these related-work systems, so it is not established elsewhere in this literature whether a credit signal’s *specificity* (as opposed to its mere sparsity) is doing the work claimed for it. This is a stronger standard than what several comparable papers report, not merely a different one; it is also, precisely because it is stronger, the reason the protocol has not yet been executed in full (Section 9).

7.1 Factor 1: Convergence Speed

Define the episodes-to-threshold metric

$$N_{\theta}(\text{method}) = \min \left\{ N : \frac{1}{100} \sum_{i=N-99}^N \mathbf{1}[\text{success}_i] \geq \theta \right\}, \quad \theta = 0.90. \quad (18)$$

VCBM and RUDDER are each trained for 5 seeds on MiniGrid-KeyCorridorS3, and the speedup ratio $N_{0.90}^{\text{RUDDER}}/N_{0.90}^{\text{VCBM}}$ is reported.

7.2 Factor 2: Performance Scaling with Horizon Length

The same comparison is repeated across MiniGrid-KeyCorridorS3 through S6 ($H \approx 30$ to $H \approx 96$), with the number of causal events held approximately constant across conditions so that $\text{CCR}(\tau)$ decreases as H increases. We define

$$\text{Gap}(H) = N_{0.90}^{\text{RUDDER}}(H) - N_{0.90}^{\text{VCBM}}(H) \quad (19)$$

and test whether $\text{Gap}(H)$ is significantly increasing in H via linear regression.

7.3 Factor 3: Robustness to Revisited States

A custom gridworld is constructed in which the agent must revisit a fixed state $\ell \in \{1, 3, 5\}$ times before the causally productive transition occurs (e.g. discovering a key on a later visit to a previously empty room). VCBM is compared against plain leave-one-out credit (no temporal or causal redistribution), since this factor tests revisit disambiguation rather than temporal redistribution and RUDDER is not the relevant baseline. We measure $\text{LoopGap}(\ell) = N_{0.90}^{\text{LOOP}}(\ell) - N_{0.90}^{\text{VCBM}}(\ell)$ and additionally report a qualitative **Loop Disambiguation Rate**: the fraction of sampled successful episodes in which the retrieved causal chain correctly identifies the productive revisit rather than an earlier, unproductive one.

7.4 Factor 4: Statistical Significance

All comparisons use a minimum of 5 random seeds. Welch’s t -test is used for pairwise comparisons of $N_{0.90}$, accompanied by Cohen’s d effect size. Linear regression coefficients for $\text{Gap}(H)$ and $\text{LoopGap}(\ell)$ are tested against the null hypothesis of zero slope. A Bonferroni correction is applied within each factor’s family of comparisons. All results are reported as mean $\pm$ standard error with associated p -values and effect sizes; no single-seed result is reported as a finding.

7.5 Ablation Design

The *LOOP-matched* condition, using identical context length, rollout count, and GPU memory allocation to VCBM, is required to attribute any performance improvement to the causal mechanism

Table 2: Ablation conditions isolating each VCBM component.

Condition	α	Detector	HTTP Tracker
LOOP-original	0.0	—	✗
LOOP-matched (identical config)	0.0	—	✗
VCBM-no-tracker	0.1	✓	✗
VCBM-random-anchors	0.1	random	✓
VCBM-uniform-credit	0.1	✓(uniform weights)	✓
VCBM-full	0.1	✓	✓

rather than to configuration differences. The *VCBM-random-anchors* condition serves as a control to test whether *specific* causal identification matters, as opposed to any sparsification of the credit signal.

8 Preliminary Empirical Observations

Before the four-factor protocol of Section 6 can be executed, it is necessary to confirm that the infrastructure the protocol depends on is itself reliable. This section reports exactly that: two completed 200-iteration training runs, used here solely to evaluate the infrastructure claims of Theorem 4 and the cost-attribution claim of Proposition 1, not the credit-assignment claim of Theorem 3, which is addressed separately in Section 9.

8.1 Training Configurations

Two independent 200-iteration training runs are reported, differing in hardware utilisation profile but using an identical base model and PPO configuration. Both use Qwen2.5-1.5B-Instruct [9] with LoRA adaptation [10] (rank 8, $\alpha_{\text{LoRA}} = 16$, 9,232,384 trainable parameters out of 1,552,946,688 total, 0.59% trainable fraction), AdamW optimisation [11] at learning rate 5×10^{-5} , PPO clip parameter $\epsilon = 0.1$, leave-one-out baseline, and an advantage-filtering threshold of 0.01. VCC-specific hyperparameters are $\alpha = 0.1$, top- $k = 5$, and softmax temperature $\tau_{\text{temp}} = 0.1$ throughout. Rollout generation in both runs uses vLLM [12] as the inference server and FSDP2 [13] for distributed gradient computation.

Configuration A (`vcc_dissertation_1_5b_training`, exclusive inference/learning mode) used a 4,096-token maximum context length, eager execution (no CUDA graph capture), 4 rollouts per iteration (2 scenarios $\times$ 2 rollouts), and explicit GPU memory partitioning between the vLLM inference server and the FSDP2 learner (12 GiB each), resulting in full process teardown and restart of the vLLM server between the rollout-collection and gradient-update phases of every iteration.

Configuration B (same experiment name, later run) used an 8,192-token maximum context length, CUDA graph capture enabled (`enforce_eager=False`, 67 graph sizes captured, graph capture overhead 0.58 GiB), `torch.compile` at optimisation level 3 with persistent compilation caching across iterations, 8 rollouts per iteration (4 scenarios $\times$ 2 rollouts), and a shared GPU memory budget (`gpu_memory_utilization=0.80`) without explicit inference/learning partitioning.

8.2 Infrastructure Robustness (Theorem 4)

Across both 200-iteration runs:

- Zero infrastructure crashes were observed across either run, against a measured $\sim 14\%$ per-iteration collapse rate in the unmodified baseline (Section 4).
- No instances of high KL divergence (policy collapse) were observed in either run; per-step `all_kl_estimate` values remained within $[-0.86, 0.78]$ throughout, consistent with stable trust-region behaviour under the PPO clipping mechanism.
- The advantage filtering fraction varied with the rollout budget: 0.75 under the 4-rollouts-per-scenario configuration and 0.88 under the 8-rollouts-per-scenario configuration, in both cases indicating substantial room for improved sample efficiency through increased rollouts per scenario, consistent with the sample-efficiency bound of Corollary 3.6.

8.3 Rollout Collection Cost Under Configuration Change

Direct `get_rollouts` wall-clock measurements from Configuration B average 14.15 s/iteration across iterations 194–199 (excluding one anomalous iteration discussed below), against an average of 36.9 s/iteration measured under an earlier, otherwise-comparable exclusive-mode configuration with eager execution and half the per-iteration rollout count. This $2.6\times$ improvement is attributable to the combined effect of CUDA graph capture, the larger KV-cache allocation (8.14 GiB versus 3.01–5.03 GiB across earlier runs), and the larger context window, despite Configuration B collecting twice as many rollouts per iteration as the comparison run.

One iteration (198) in Configuration B measured 48.96 s for `get_rollouts`, of which 39.03 s was attributable to a single episode (`ccb4494_1`); this is consistent with ordinary variance in generation length across sampled tasks rather than the infrastructure-collapse pattern characterised in Section 4.1, since execution completed normally and no exception or forced server restart occurred.

8.4 Relation to the Theoretical Claims

These results support Theorem 4 and its corollary directly, under two materially different hardware and rollout-budget configurations. The infrastructure robustness result is configuration-independent: zero collapses were observed regardless of whether eager or compiled execution was used, or whether 4 or 8 rollouts were collected per iteration, consistent with the bound in Theorem 4 depending only on $T_{\max}$, T_{gen} , and T_{overhead} and not on the surrounding training configuration. The results of Section 5 do not yet constitute evidence for Theorem 3, which requires the controlled LOOP-matched ablation and the four-factor protocol of Section 6; they do, however, directly substantiate the cost-attribution claim of Proposition 1, since both VCBM runs reported here include the naive state-diffing detector design whose overhead that proposition quantifies.

9 Limitations

This work has five limitations, each bounding a different claim made above.

Detector granularity. The causal event detector’s accuracy is bounded by the granularity of the state-change function f ; false negatives introduce bias proportional to the contribution of missed causal events (Proposition 4.1, omitted here for brevity but available in the supplementary theoretical document).

Baseline reuse. The current implementation reuses the leave-one-out baseline rather than training a dedicated value network, which may understate VCBM’s achievable variance reduction relative to the bound proved in Theorem 3.

RUDDER comparison scope. RUDDER is evaluated only in MiniGrid, since adapting its LSTM architecture to natural-language API trajectories without a confounding redesign was judged infeasible within the project timeline; the convergence-speed and horizon-scaling comparisons of Sections 6.1–6.2 should therefore be read as evidence about the gridworld setting specifically, not as a direct claim about RUDDER’s behaviour on AppWorld-scale trajectories.

Unconfirmed detector speedup. The HTTP-method detector of Section 5.3 has been specified and its equivalence to the state-diffing detector established by Proposition 2, but the projected 16–17 $\times$ rollout-collection speedup has not yet been confirmed by a re-run of the full 200-iteration ablation.

No direct empirical comparison against the closest learned baseline. Section 4.2 identifies Memory-R2’s LoGo-GRPO [27] as the prior mechanism architecturally closest to VCBM’s credit weighting, since both apply a form of per-step gating on top of a group-relative advantage. The argument that VCBM’s hard, detector-based gate is preferable to LoGo-GRPO’s learned gate is made in Section 4.2 on structural grounds (no approximation error from an auxiliary network, a formal unbiasedness guarantee that a learned gate does not have) and is not yet supported by a head-to-head experimental comparison; the four-factor protocol of Section 6 evaluates VCBM against RUDDER and plain leave-one-out credit, not against LoGo-GRPO or the other learned-gate baselines discussed in Related Work, so the structural argument for VCBM’s advantage over the closest prior mechanism remains theoretical until such a comparison is run.

Taken together, the re-measurement of rollout-collection cost under the corrected detector and the four-factor protocol of Section 6 are both required before the central variance-reduction claim of Theorem 3 can be considered empirically validated rather than theoretically established; a further, separate comparison against LoGo-GRPO and related learned-gate baselines would additionally be required before VCBM’s advantage over the closest prior mechanism, currently argued only on structural grounds, could be considered empirically established.

10 Conclusion

This paper addressed the credit-assignment problem in long-horizon agentic reinforcement learning by identifying *which* timesteps are causally responsible for a sparse terminal reward, rather than only *when* credit is redistributed relative to that reward. The core idea, Visual Causal Chain Bookmarking, detects causally active timesteps directly from persistent state change, retrieves the subset most relevant to the observed outcome, and restricts the policy gradient update to that subset; we proved this estimator unbiased and variance-reducing relative to the uniform credit assignment used by LOOP and GRPO.

Beyond this central mechanism, we established a deterministic safety bound on rollout collection time under explicit API request timeout enforcement, and a four-factor experimental protocol designed to produce falsifiable evidence for convergence speed, horizon scaling, revisit robustness, and statistical significance.

We have further shown, via direct wall-clock instrumentation across two independent 200-iteration training configurations, that the infrastructure robustness guarantee of Theorem 4 holds in practice independent of hardware configuration, that a naive implementation of the causal event detector is responsible for a 1596% rollout-collection overhead concentrated almost entirely (98.0%) in state-snapshot diffing, and that a zero-extra-call HTTP-method-based detector is extensionally equivalent to the naive detector under the REST semantics of the target environment.

These preliminary results confirm the infrastructure-robustness claim and the overhead-attribution claim, and no more: the central credit-assignment claim of Theorem 3 remains theoretically estab-

lished but not yet empirically validated. Confirming it is the immediate next step, and requires two things not yet performed — the controlled LOOP-matched ablation and horizon-scaling experiments of Section 6, and a re-measurement of rollout-collection cost under the corrected detector specified in Section 5.3 to confirm the projected speedup reported there.

References

- [1] J. A. Arjona-Medina, M. Gillhofer, M. Widrich, T. Unterthiner, J. Brandstetter, and S. Hochreiter. *RUDDER: Return Decomposition for Delayed Rewards*. NeurIPS, 2019.
- [2] J. Schulman, P. Moritz, S. Levine, M. Jordan, and P. Abbeel. *High-Dimensional Continuous Control Using Generalized Advantage Estimation*. ICLR, 2016.
- [3] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov. *Proximal Policy Optimization Algorithms*. arXiv:1707.06347, 2017.
- [4] Apple ML Research. *LOOP: Leave-One-Out Policy Optimization for Long-Horizon Agentic Tasks*. 2025.
- [5] DeepSeek-AI. *Group Relative Policy Optimization*. 2024.
- [6] M. Andrychowicz, F. Wolski, A. Ray, et al. *Hindsight Experience Replay*. NeurIPS, 2017.
- [7] A. Pritzel, B. Uria, S. Srinivasan, et al. *Neural Episodic Control*. ICML, 2017.
- [8] H. Trivedi, T. Khot, M. Hartmann, R. Manku, V. Dong, E. Li, S. Gupta, A. Sabharwal, and N. Balasubramanian. *AppWorld: A Controllable World of Apps and People for Benchmarking Interactive Coding Agents*. ACL, 2024.
- [9] Qwen Team, Alibaba Group. *Qwen2.5 Technical Report*. arXiv:2412.15115, 2024.
- [10] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen. *LoRA: Low-Rank Adaptation of Large Language Models*. arXiv:2106.09685, 2021.
- [11] I. Loshchilov and F. Hutter. *Decoupled Weight Decay Regularization*. ICLR, 2019.
- [12] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. Gonzalez, H. Zhang, and I. Stoica. *Efficient Memory Management for Large Language Model Serving with PagedAttention*. SOSP, 2023.
- [13] Y. Zhao, A. Gu, R. Varma, L. Luo, C. C. Huang, M. Xu, L. Wright, H. Shojanazeri, M. Ott, S. Shleifer, A. Desmaison, C. Balioglu, P. Damania, B. Nguyen, G. Chauhan, Y. Hao, A. Mathews, and S. Li. *PyTorch FSDP: Experiences on Scaling Fully Sharded Data Parallel*. VLDB, 2023.
- [14] Pignatelli et al. **[TITLE/VENUE/YEAR NEEDED]** *Survey of temporal credit assignment in deep reinforcement learning, providing a unifying formalism and evaluation protocols*. **[TODO – FULL CITATION INCOMPLETE, verify against source before submission]**
- [15] Harutyunyan et al. **[TITLE/VENUE/YEAR NEEDED]** *Hindsight Credit Assignment: assigns credit to past actions using a hindsight distribution conditioned on outcomes*. **[TODO – FULL CITATION INCOMPLETE, verify against source before submission]**

- [16] Mesnard et al. [TITLE/VENUE/YEAR NEEDED] *Counterfactual Credit Assignment: uses structural causal models to estimate counterfactual returns in RL.* [TODO – FULL CITATION INCOMPLETE, verify against source before submission]
- [17] [AUTHORS NEEDED]. [TITLE/VENUE/YEAR NEEDED] *COCOA: counterfactual contribution analysis for long-term credit assignment via model-based counterfactual queries.* [TODO – FULL CITATION INCOMPLETE, verify against source before submission]
- [18] [AUTHORS NEEDED]. [TITLE/VENUE/YEAR NEEDED] *LVGTS: model-based temporal shortcuts to propagate gradients from future rewards back to distant past decisions.* [TODO – FULL CITATION INCOMPLETE, verify against source before submission]
- [19] Ren et al. [TITLE/VENUE/YEAR NEEDED] *Learning long-term reward redistribution via randomized return decomposition for delayed rewards.* [TODO – FULL CITATION INCOMPLETE, verify against source before submission]
- [20] [AUTHORS NEEDED]. [TITLE/VENUE/YEAR NEEDED] *Counterfactual Shapley Credit Assignment (ϕ -PPO): Shapley values to redistribute rewards across sparse, stochastic trajectories.* [TODO – FULL CITATION INCOMPLETE, verify against source before submission]
- [21] [AUTHORS NEEDED]. [TITLE/VENUE/YEAR NEEDED] *Agentic RL survey emphasizing credit assignment as a central bottleneck in scaling LLM agents.* [TODO – FULL CITATION INCOMPLETE, verify against source before submission]
- [22] [AUTHORS NEEDED]. [TITLE/VENUE/YEAR NEEDED] *Survey organizing credit-assignment methods for LLM agents by granularity and methodology, emphasizing long-horizon challenges.* [TODO – FULL CITATION INCOMPLETE, verify against source before submission]
- [23] [AUTHORS NEEDED]. [TITLE/VENUE/YEAR NEEDED] *Survey on long-horizon agents reviewing hierarchical and credit-assignment techniques for manipulation and LLM tasks.* [TODO – FULL CITATION INCOMPLETE, verify against source before submission]
- [24] [AUTHORS NEEDED]. [TITLE/VENUE/YEAR NEEDED] *Empirical study of how horizon length affects training large language models for long-horizon tasks.* [TODO – FULL CITATION INCOMPLETE, verify against source before submission]
- [25] [AUTHORS NEEDED]. [TITLE/VENUE/YEAR NEEDED] *Multi-turn GRPO variants: a framework for turn-level credit assignment in multi-step reasoning.* [TODO – FULL CITATION INCOMPLETE, verify against source before submission]
- [26] [AUTHORS NEEDED]. [TITLE/VENUE/YEAR NEEDED] *Hybrid Advantage Shaping with TurnRD: combines GRPO with learned per-turn credit for multi-turn LLM agents.* [TODO – FULL CITATION INCOMPLETE, verify against source before submission]
- [27] [AUTHORS NEEDED]. [TITLE/VENUE/YEAR NEEDED] *Memory-R2: fair credit assignment for long-horizon memory-augmented LLM agents via LoGo-GRPO and curriculum*

training. [TODO – FULL CITATION INCOMPLETE, verify against source before submission]

- [28] [AUTHORS NEEDED]. [TITLE/VENUE/YEAR NEEDED] *Ragen: self-evolution in LLM agents via multi-turn RL, analyzing credit assignment across reasoning turns.* [TODO – FULL CITATION INCOMPLETE, verify against source before submission]
- [29] [AUTHORS NEEDED]. [TITLE/VENUE/YEAR NEEDED] *ProRL: expands RL training horizons for LLM reasoning, examining how prolonged interaction affects credit assignment.* [TODO – FULL CITATION INCOMPLETE, verify against source before submission]
- [30] [AUTHORS NEEDED]. [TITLE/VENUE/YEAR NEEDED] *Memory-augmented credit assignment frameworks handling shared memory states in long-horizon LLM agents.* [TODO – FULL CITATION INCOMPLETE, verify against source before submission]
- [31] Tajwar et al. [TITLE/VENUE/YEAR NEEDED] *Training a generally curious agent, using curiosity signals to guide long-horizon exploration and credit assignment.* [TODO – FULL CITATION INCOMPLETE, verify against source before submission]
- [32] Auzina et al. [TITLE/VENUE/YEAR NEEDED] *Δ Belief-RL: changes in the model’s own belief as intrinsic rewards for long-horizon credit assignment in LLM interaction tasks.* [TODO – FULL CITATION INCOMPLETE, verify against source before submission]
- [33] [AUTHORS NEEDED]. [TITLE/VENUE/YEAR NEEDED] *OpenSearch-VL: process reward and credit-assignment techniques in multimodal search agents.* [TODO – FULL CITATION INCOMPLETE, verify against source before submission]
- [34] [AUTHORS NEEDED]. [TITLE/VENUE/YEAR NEEDED] *InfoPO: optimizing user-centric agents by assigning credit based on information gain along interaction trajectories.* [TODO – FULL CITATION INCOMPLETE, verify against source before submission]
- [35] [AUTHORS NEEDED]. [TITLE/VENUE/YEAR NEEDED] *ICA: information-aware credit assignment for visually grounded long-horizon web agents using visual snapshots and posterior analysis.* [TODO – FULL CITATION INCOMPLETE, verify against source before submission]
- [36] [AUTHORS NEEDED]. [TITLE/VENUE/YEAR NEEDED] *Evaluate-as-Action: self-evaluation as an action yielding process rewards for retrieval-augmented agents.* [TODO – FULL CITATION INCOMPLETE, verify against source before submission]
- [37] [AUTHORS NEEDED]. [TITLE/VENUE/YEAR NEEDED] *PRAISE: reuses search prefixes and derives step-level process rewards from prefix gains to improve credit assignment in agentic search.* [TODO – FULL CITATION INCOMPLETE, verify against source before submission]
- [38] [AUTHORS NEEDED]. [TITLE/VENUE/YEAR NEEDED] *Process Reward Agents: intermediate rewards to steer knowledge-intensive LLM agents, improving process-level credit.* [TODO – FULL CITATION INCOMPLETE, verify against source before submission]

- [39] **[AUTHORS NEEDED]. [TITLE/VENUE/YEAR NEEDED]** *SubSearch: intermediate rewards for unsupervised guided reasoning in retrieval-augmented agents.* **[TODO – FULL CITATION INCOMPLETE, verify against source before submission]**